

THE UNIVERSITY OF MELBOURNE

COMP90038 Algorithms and Complexity

Lecture 13: Priority Queues, Heaps and Heapsort

Olya Ohrimenko

(with thanks to Harald Søndergaard)

Priority Queues

THE UNIVERSITY OF MELBOURNE

A priority queue is a set (or pool) of elements.

An element is injected into the priority queue together with a priority (often the key value itself) and elements are ejected according to priority.

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

The Priority Queue

THE UNIVERSITY OF MELBOURNE

As an abstract data type, the priority queue supports the following operations on a "pool" of elements (ordered by some linear order):

- find an item with maximal priority
- insert a new item with associated priority
- test whether a priority queue is empty
- eject the largest element

Other operations may be relevant, for example:

- replace the maximal item with some new item
- construct a priority queue from a list of items
- join two priority queues

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Stacks and Queues as Priority Queues

THE UNIVERSITY OF MELBOURNE

Special instances are obtained when we use time for priority:

- If “large” means “late” we obtain the stack.
- If “large” means “early” we obtain the queue.

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Some Uses of Priority Queues

THE UNIVERSITY OF MELBOURNE

- Job scheduling done by your operating system. The OS will usually have a notion of "importance" of different jobs.
- Traffic management (e.g., arrival times of network requests)
- (Discrete event) simulation of complex systems (like traffic, or weather). Here priorities are typically event times.

Many sophisticated algorithms make essential use of priority queues (Huffman encoding and many shortest-path algorithms, for example).

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

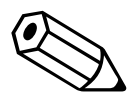
Possible Implementations of the Priority Queue

THE UNIVERSITY OF MELBOURNE

Assume priority = key.

	Inject(e)	Eject()
Unsorted array or list		
Sorted array or list		

How is this accomplished?



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

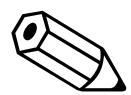
Possible Implementations of the Priority Queue

THE UNIVERSITY OF MELBOURNE

Assume priority = key.

	Inject(e)	Eject()
Unsorted array or list		
Sorted array or list		
Heap	$O(\log n)$	$O(\log n)$

How is this accomplished?



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Heaps (High Level)

THE UNIVERSITY OF MELBOURNE

The heap is a very useful data structure for priority queues, used in many algorithms.

We think of the heap as a partially ordered binary tree.

Since it can easily be maintained as a complete tree, the standard implementation uses an array to represent the tree.

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

The Heap

THE UNIVERSITY OF MELBOURNE

A heap is a complete binary tree which satisfies the heap condition:

Each child has a priority (key) which is no greater than its parent's.

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

The Heap

THE UNIVERSITY OF MELBOURNE

A heap is a complete binary tree which satisfies the heap condition:

Each child has a priority (key) which is no greater than its parent's.

This guarantees that the root of the tree is a maximal element.

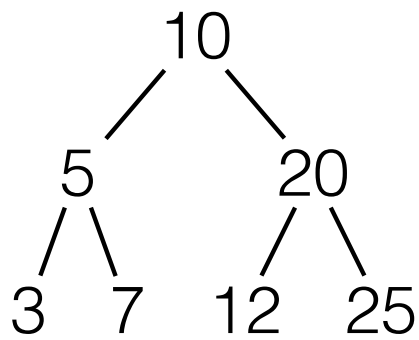
(Sometimes we talk about this as a max-heap—one can equally well have min-heaps, in which each child is no smaller than its parent.)

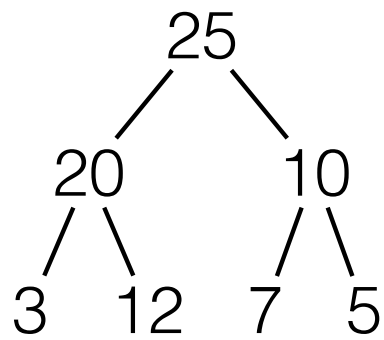
Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

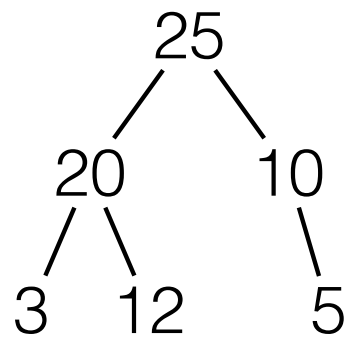
Heaps and Non-Heaps

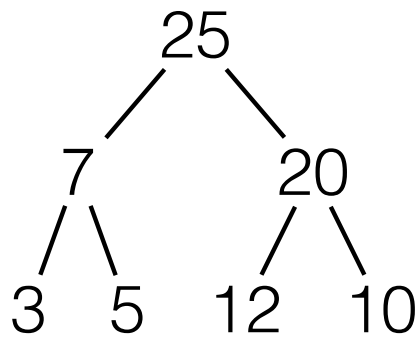
THE UNIVERSITY OF MELBOURNE

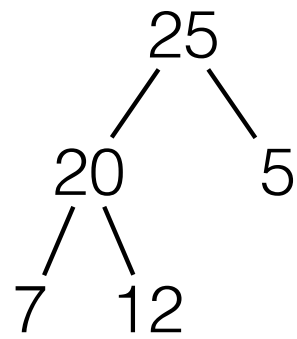
Which of these are heaps?

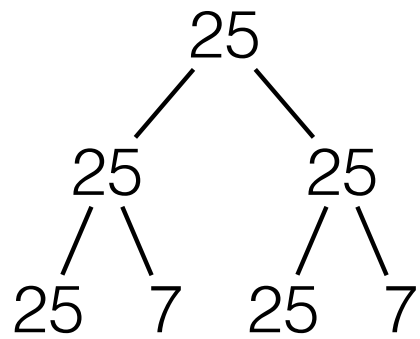










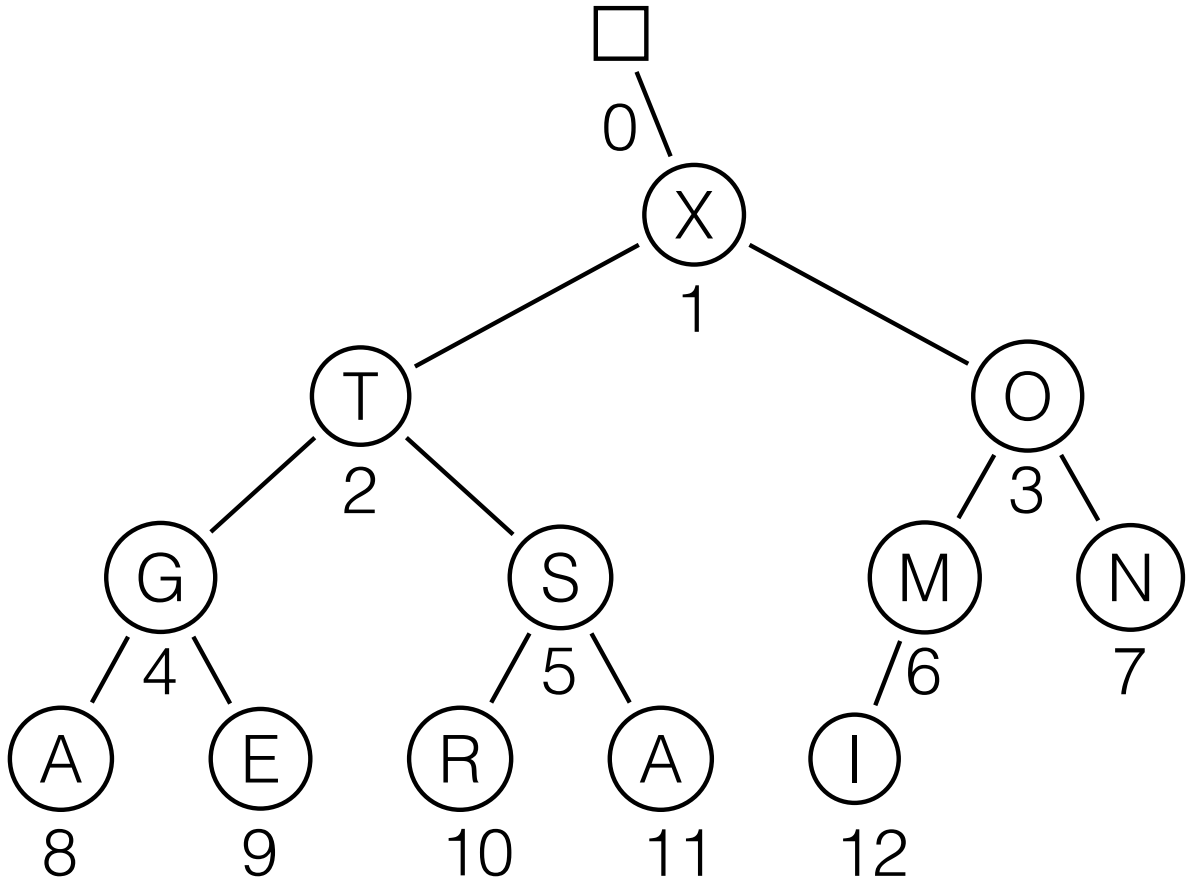


Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Heaps as Arrays

We can utilise the completeness of the tree and place its elements in level-order in an array H.

Note that the children of node i will be nodes $2i$ and $2i + 1$.



H:

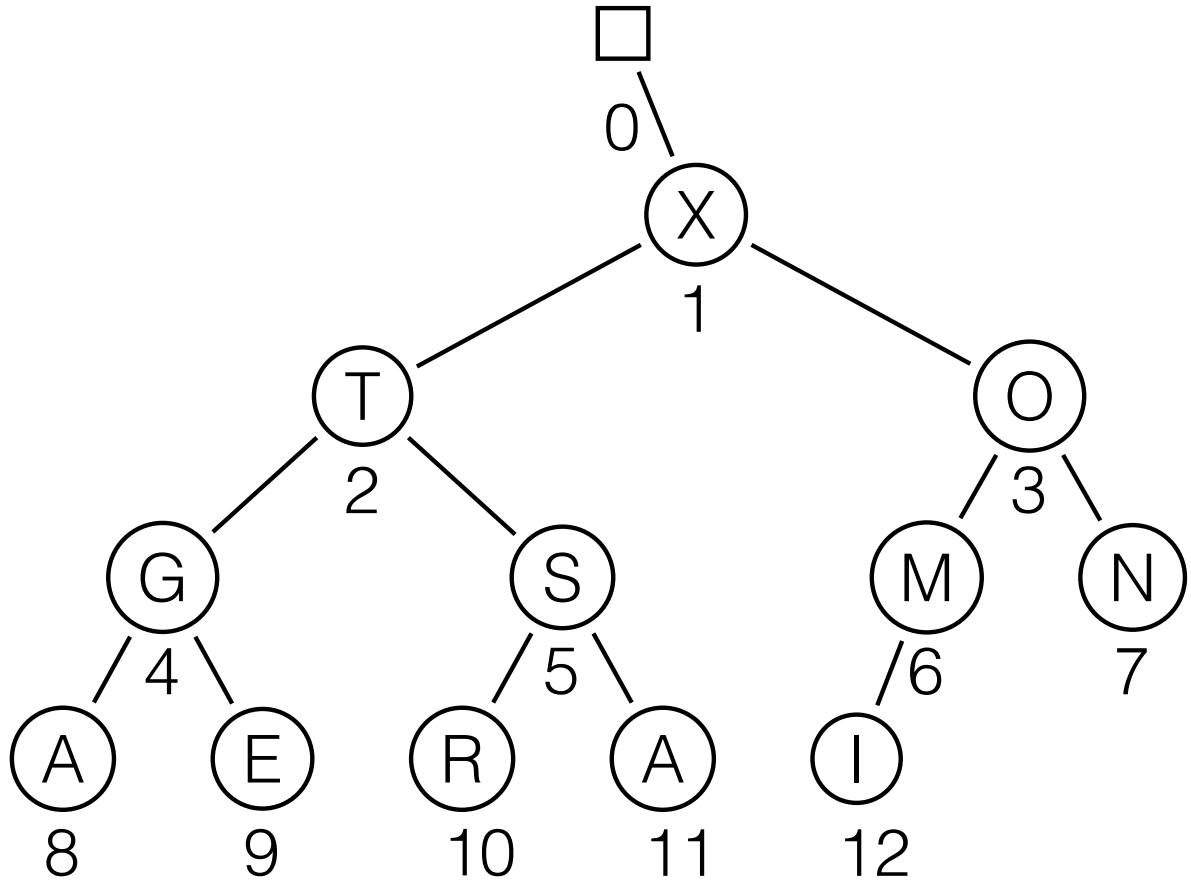
	X	T	O	G	S	M	N	A	E	R	A	I
0	1	2	3	4	5	6	7	8	9	10	11	12

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Heaps as Arrays

This way, the heap condition is very simple:

For all i $\in \{0, 1, \ldots, n\}$, we must have $H[i] \leq H[i/2]$.



H:

	X	T	O	G	S	M	N	A	E	R	A	I
0	1	2	3	4	5	6	7	8	9	10	11	12

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Properties of the Heap

The root of the tree $H[1]$ holds a maximal item; the cost of Eject is $O(1)$ plus time to restore the heap.

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Properties of the Heap

The root of the tree $H[1]$ holds a maximal item; the cost of Eject is $O(1)$ plus time to restore the heap.

The height of the heap is $\lfloor \log_2 n \rfloor$.

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Properties of the Heap

The root of the tree $H[1]$ holds a maximal item; the cost of Eject is $O(1)$ plus time to restore the heap.

The height of the heap is $\lfloor \log_2 n \rfloor$.

Each subtree is also a heap.

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Properties of the Heap

THE UNIVERSITY OF MELBOURNE

The root of the tree H[1] holds a maximal item; the cost of Eject is O(1) plus time to restore the heap.

The height of the heap is $\lfloor \log_2 n \rfloor$.

Each subtree is also a heap.

The children of node i are $2i$ and $2i + 1$.

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Properties of the Heap

THE UNIVERSITY OF MELBOURNE

The root of the tree H[1] holds a maximal item; the cost of Eject is O(1) plus time to restore the heap.

The height of the heap is $\lfloor \log_2 n \rfloor$.

Each subtree is also a heap.

The children of node i are $2i$ and $2i + 1$.

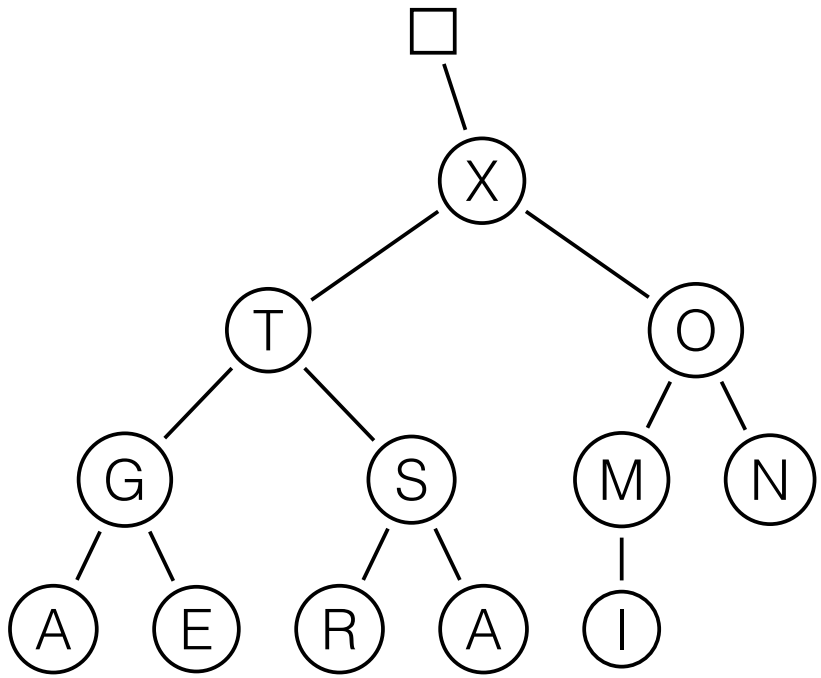
The nodes which happen to be parents are in array positions 1 to $\lfloor n/2 \rfloor$.

It is easier to understand the heap operations if we think of the heap as a tree.

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Injecting a New Item

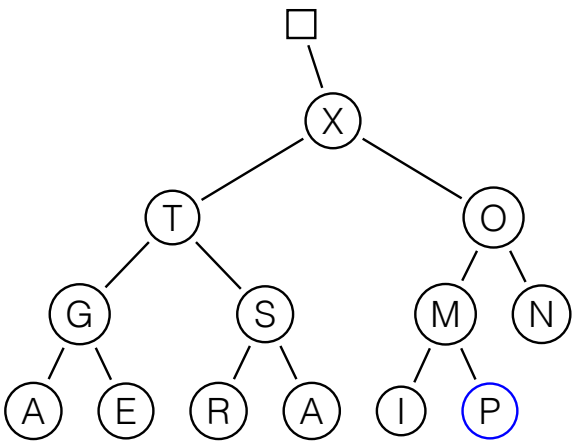
Place the new item at the end; then let it “climb up”, repeatedly swapping with parents that are smaller:



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Injecting a New Item

Place the new item at the end; then let it “climb up”, repeatedly swapping with parents that are smaller:



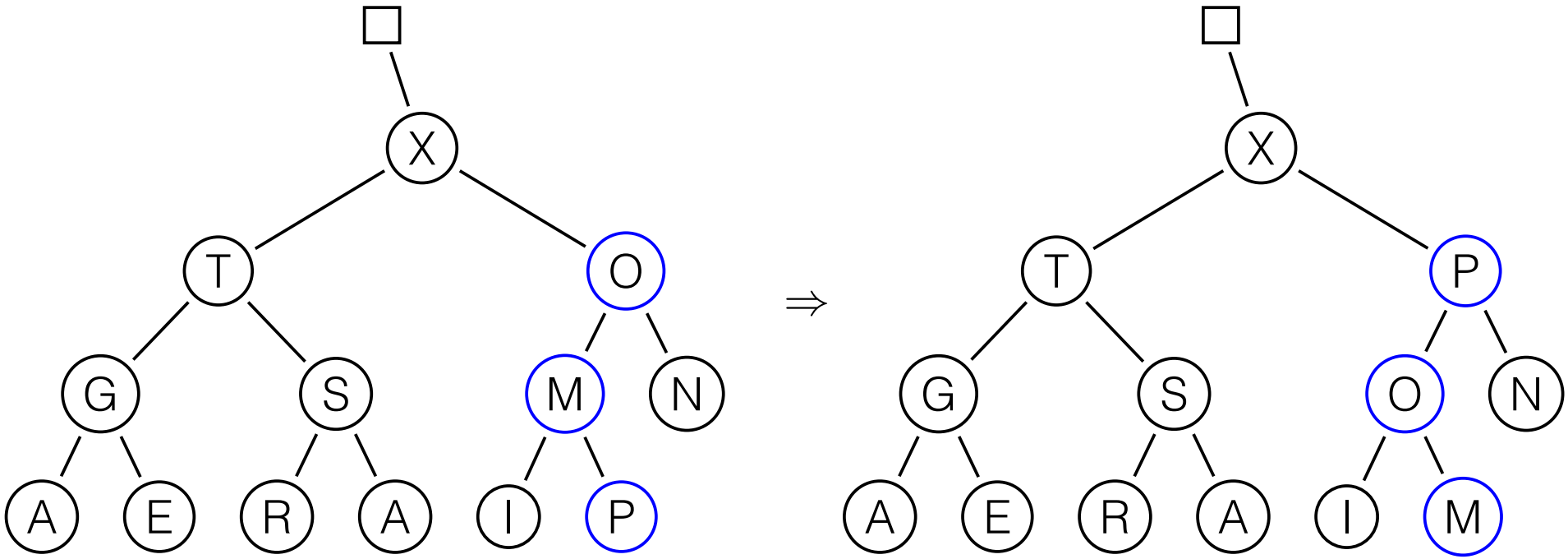
Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Left blank

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Injecting a New Item

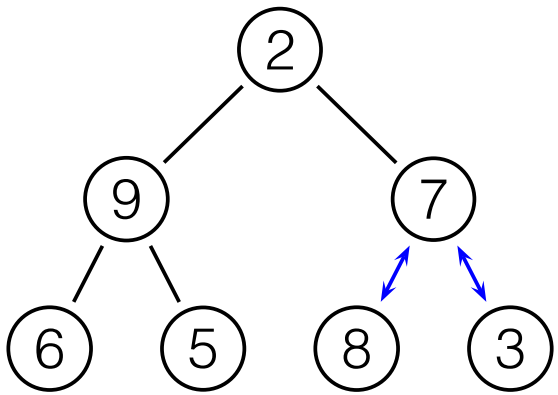
Place the new item at the end; then let it “climb up”, repeatedly swapping with parents that are smaller:



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Building a Heap Bottom-Up

To construct a heap from an arbitrary set of elements, we can just use the inject operation repeatedly. The construction cost will be $n \log n$. But there is a better way:



Start with the last parent and move backwards, in level-order. For each parent node, if the largest child is larger than the parent, swap it with the parent.

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

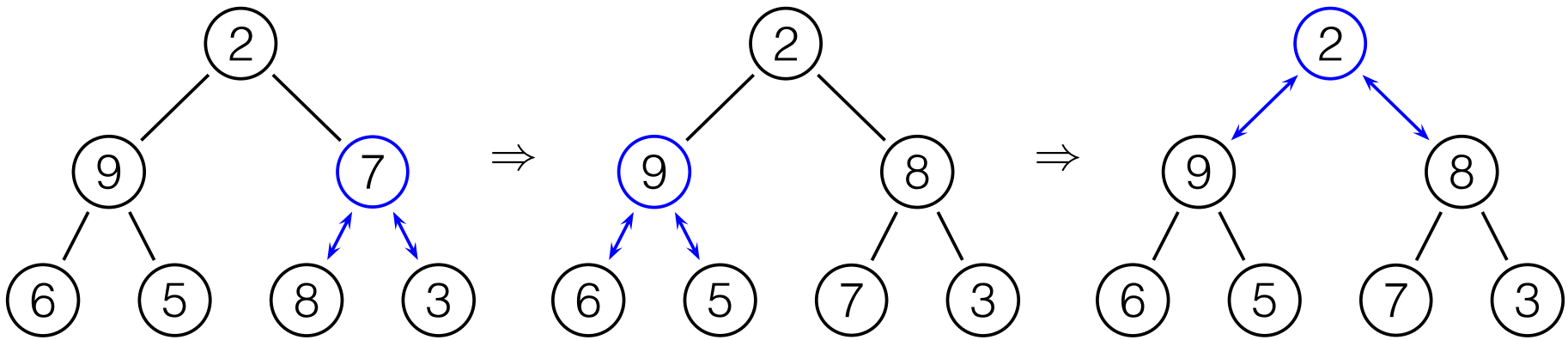
Left blank

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Building a Heap Bottom-Up

THE UNIVERSITY OF MELBOURNE

To construct a heap from an arbitrary set of elements, we can just use the inject operation repeatedly. The construction cost will be $n \log n$. But there is a better way:



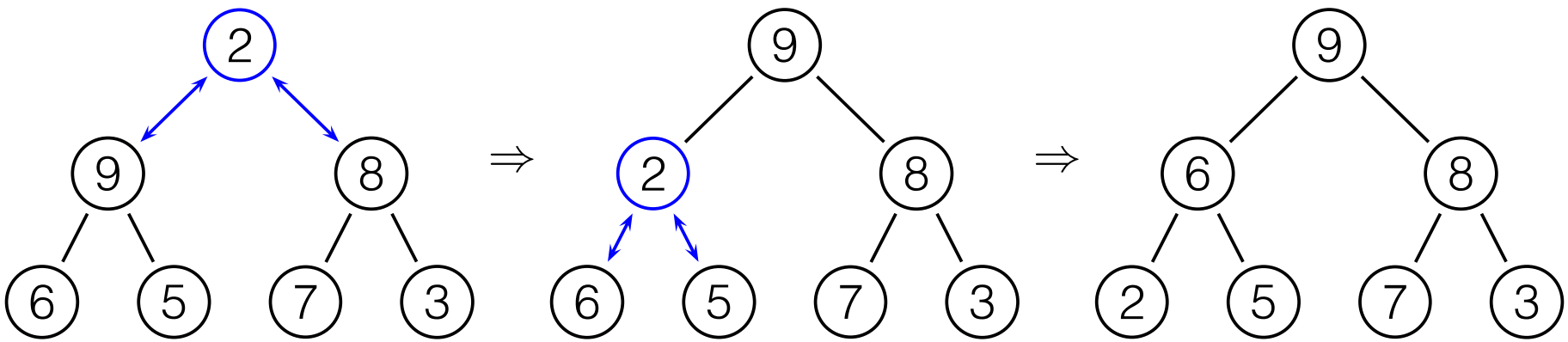
Start with the last parent and move backwards, in level-order. For each parent node, if the largest child is larger than the parent, swap it with the parent.

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Building a Heap Bottom-Up: Sifting Down

THE UNIVERSITY OF MELBOURNE

Whenever a parent is found to be out of order, let it “sift down” until both children are smaller:



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Turn H into a Heap: Bottom-Up

```
for i ← ⌊n/2⌋ downto 1 do
    k ← i
    v ← H[k]
    heap ← False
    while not heap and 2 × k ≤ n do
        j ← 2 × k                                  ▷ j is k's left child
        if j < n then
            if H[j] < H[j + 1] then
                j ← j + 1                          ▷ j is k's largest child
        if v ≥ H[j] then
            heap ← True
        else                                       ▷ Promote H[j]
            H[k] ← H[j]
            k ← j
    H[k] ← v
```

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Analysis of Bottom-Up Heap Creation

THE UNIVERSITY OF MELBOURNE

For simplicity, assume the heap is a full binary tree:
$n = 2^{h+1} - 1$.
Here is an upper bound on the number of "down-sifts" needed (consider the root to be at level h, so leaves are at level 0):

$$\sum_{i=1}^{h} \sum_{\text{nodes at level } i} i = \sum_{i=1}^{h} i \cdot 2^{h-i} = 2^{h+1} - h - 2$$

The last equation is easily proved by mathematical induction.

Note that $2^{h+1} - h - 2 < n$, so we perform at most a linear number of down-sift operations. Each down-sift is preceded by two key comparisons, so the # comparisons is also linear.

Hence we have a linear-time algorithm for heap creation.

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Left blank

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

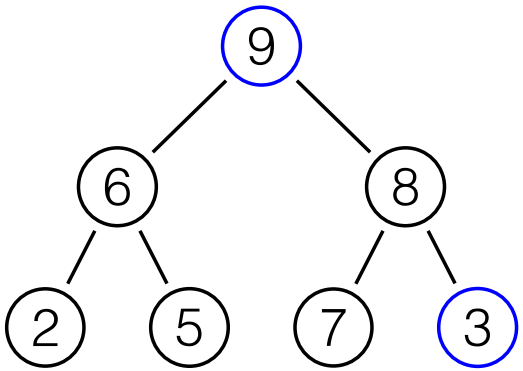
Useful formulas

$$\sum_{i=1}^{n} i2^i = 1 \cdot 2 + 2 \cdot 2^2 + \cdots + n2^n = (n-1)2^{n+1} + 2$$

$$\sum_{i=0}^{n} 2^i = 2^{n+1} - 1$$

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Ejecting a Maximal Element from a Heap: Example



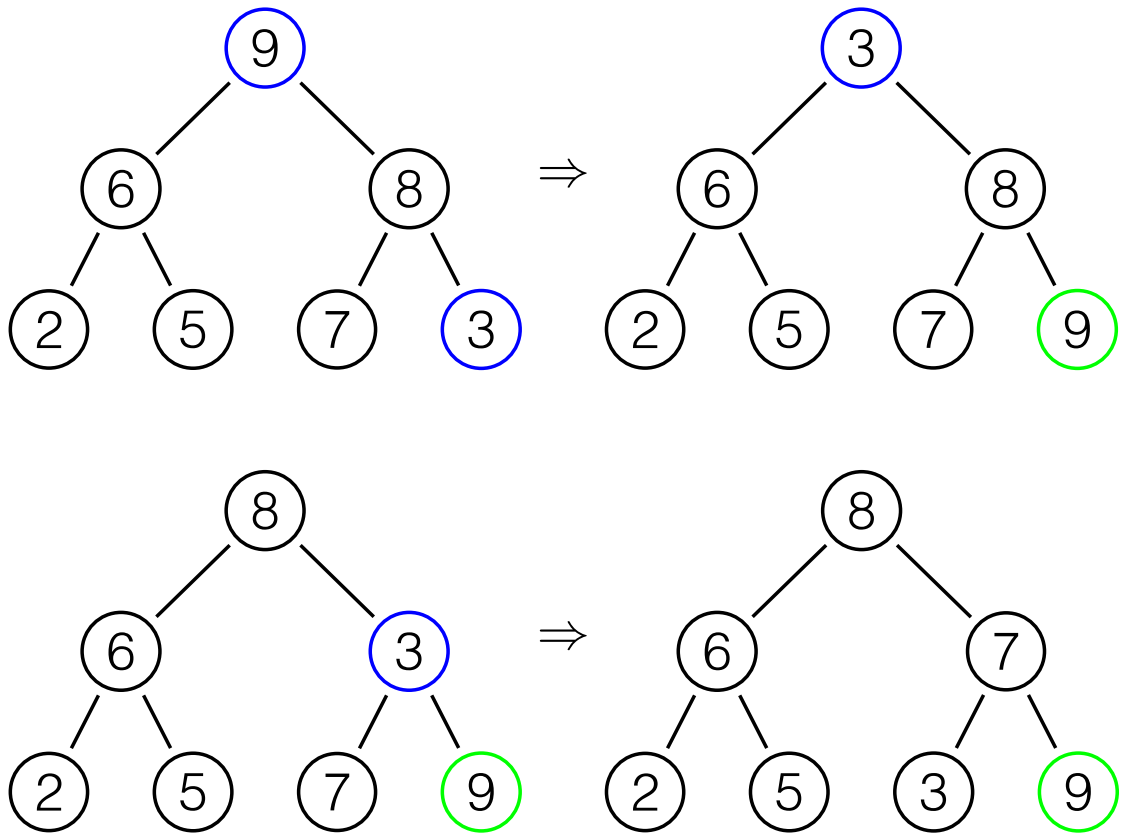
Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Ejecting a Maximal Element from a Heap

Here the idea is to swap the root with the last item z in the heap, and then let z "sift down" to its proper place.

After this, the last element (here shown in green) is no longer considered part of the heap, that is, n is decremented.

Ejection is $O(\log n)$.



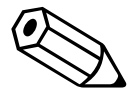
Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Exercise: Build and Then Deplete a Heap

THE UNIVERSITY OF MELBOURNE

First build a heap from the items S, O, R, T, I, N, G.

Then repeatedly eject the largest, placing it at the end of the heap.



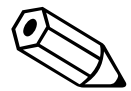
Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Exercise: Build and Then Deplete a Heap

THE UNIVERSITY OF MELBOURNE

First build a heap from the items S, O, R, T, I, N, G.

Then repeatedly eject the largest, placing it at the end of the heap.



Anything interesting to notice about the tree that used to hold a heap?

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Heapsort

Heapsort is a $\Theta(n \log n)$ sorting algorithm, based on the idea from this exercise.

Given an unsorted array $H[1] \ldots H[n]$:

Step 1 Turn H into a heap.

Step 2 Apply the eject operation $n - 1$ times.

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Properties of Heapsort

THE UNIVERSITY OF MELBOURNE

On average slower than quicksort, but stronger performance guarantee.

Truly in-place.

Not stable.

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Summary & Next Lecture

THE UNIVERSITY OF MELBOURNE

This lecture: Priority Queues via Heaps; Heap operations and algorithms

Next lecture: Tranform-and-Conquer, more on trees.

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License